

Chapter 6 · Lesson 1 — Recharts Fundamentals

Chapter 6 · Lesson 1 — Recharts Fundamentals

*Goal: understand the **mental model** behind Recharts so the actual API feels predictable. By the end of this lesson, you can read any Recharts code in the wild and describe what it draws without running it.*

The single insight that makes Recharts click

Recharts is a **declarative** chart library. Declarative means: you describe the *result you want*, not the steps to draw it. You hand Recharts an array of objects and a couple of `<Bar>/<Line>/<Pie>` declarations, and it figures out pixels, axes, scales, animations.

The key insight: **a chart is a function of the data array's shape.**

Specifically:

- The **rows** of the array become the X-axis categories (one bar per row, one slice per row, etc.).
- The **fields** of each row become the data series. Want one bar per row? Use one field. Want stacked bars? Use multiple fields.

Get the data shape right, and the chart writes itself. Get it wrong, and no amount of `<Bar>` props will save you. **90% of charting work is reshaping data.** This lesson sets up that mindset; Lessons 2 and 3 do it for real.

The shape of a Recharts chart

Open `components/tabs/AnalyticsTab.tsx`. The simplest chart in there:

```
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts';

<ResponsiveContainer width="100%" height={360}>
  <BarChart data={r1Data}>
    <CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
    <XAxis dataKey="team" />
    <YAxis />
    <Tooltip />
    <Legend />
    <Bar dataKey="R1" stackId="a" fill="#0F5132" name="Round 1" />
    <Bar dataKey="R2" stackId="a" fill="#DC2626" name="Round 2" />
    <Bar dataKey="QF" stackId="a" fill="#FBBF24" name="Quarter-Finals" />
    <Bar dataKey="SF" stackId="a" fill="#7C3AED" name="Semi-Finals" />
    <Bar dataKey="Final" stackId="a" fill="#9F1239" name="Final" />
  </BarChart>
</ResponsiveContainer>
```

Read it like a sentence: *“Inside a responsive container, draw a bar chart of r1Data. Use team as the X-axis. Five <Bar> series, all stacked together, each reading a different field of the data.”*

That’s the API. **The data is r1Data. The X-axis label is team. The Y values are R1, R2, QF, SF, Final.** Recharts handles the rest.

The data prop is a flat array of objects

r1Data looks like this (one row per team):

```
[
  { team: 'Invincibles', R1: 35, R2: 19, QF: 9, SF: 5, Final: 3, Total: 71 },
  { team: 'Uncredibles', R1: 29, R2: 15, QF: 6, SF: 3, Final: 1, Total: 54 },
  // ...
]
```

Six fields per row. team is the categorical (X-axis). The rest are numeric (Y-values).

When <Bar dataKey="R1"> evaluates, Recharts walks every row and pulls row.R1 to draw a bar. Then <Bar dataKey="R2"> does the same for R2 — but because they share

stackId="a", the R2 bar is stacked on top of the R1 bar. Same for QF, SF, Final.

One pass through the data per <Bar>. That's the whole pattern.

The <ResponsiveContainer> is non-negotiable

```
<ResponsiveContainer width="100%" height={360}>
  <BarChart>...</BarChart>
</ResponsiveContainer>
```

Without <ResponsiveContainer>, Recharts charts have **no size**. They render as zero-pixel-wide invisible elements and you spend an hour wondering why your screen is blank.

<ResponsiveContainer> is React's "fill my parent" wrapper for SVG-based charts. It listens for resize events and tells the chart what dimensions to draw at. Always wrap your charts in it.

The two props:

- width="100%" — fills the parent's width.
- height={360} — fixed height (Recharts charts can't auto-size their own height; you have to specify).

You can also use aspect={2} instead of height, for a 2:1 width:height ratio. Either works.

The four "decoration" components

```
<CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
<XAxis dataKey="team" />
<YAxis />
<Tooltip />
<Legend />
```

These show up on almost every Recharts chart. Walk through them.

<CartesianGrid> — the dotted/dashed grid

Draws faint lines across the chart for visual reference. The strokeDasharray="3 3" says "3px line, 3px gap" — i.e. dotted/dashed.

You can omit it on simple charts. Most production charts have one because it makes values easier to read.

<XAxis> – the X-axis

The `dataKey="team"` is the **categorical key** – Recharts looks at each row's team field to label the axis. If you forget `dataKey`, Recharts uses array indices (0, 1, 2, ...), which is rarely what you want.

You can also style ticks: `<XAxis tick={{ fontSize: 12, fill: '#1F2937' }}>` for smaller, darker labels.

<YAxis> – the Y-axis

Recharts auto-computes the domain (min/max) from your data. If you want to force it: `domain={[0, 100]}`. Useful for percentages where 100 is the natural max.

<Tooltip> – the hover tooltip

When the user hovers a bar, this floating box shows the values. Customizable via `<Tooltip contentStyle={{...}}>` or, for full control, a custom `content={MyCustomTooltip}` render prop.

The defaults are usually fine for a dashboard. Don't over-engineer the tooltip on day one.

<Legend> – the color key below

Shows which color = which series. Generated automatically from `<Bar name="Round 1">` props.

You can position it: `<Legend verticalAlign="top">`. Default is bottom.

Tree-shakable imports

```
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveCont
```

You import **only the components you use**. The Recharts library has dozens of components – `LineChart`, `PieChart`, `AreaChart`, `RadarChart`, `ScatterChart`, etc. – but if you don't import them, they don't end up in your bundle.

This is why Recharts can have a “huge” surface area without producing a huge bundle. **Tree-shaking + named imports = small bundle.** Modern bundlers (the one Next.js uses, SWC) shake aggressively.

The chart card pattern

In our codebase, every chart is wrapped in a `<ChartCard>`:

```
// components/analytics/ChartCard.tsx
interface ChartCardProps {
  title: string;
  subtitle: string;
  children: React.ReactNode;
}

export default function ChartCard({ title, subtitle, children }: ChartCardProps) {
  return (
    <div style={{
      background: '#FFFFFF',
      borderRadius: 16,
      padding: 28,
      boxShadow: '0 4px 20px rgba(0,0,0,0.06)',
      border: '2px solid #FEF3C7',
    }}>
      <h3 style={{ fontFamily: "'Roboto Slab', serif", fontSize: 22, color:
        {title}
      }}>
      </h3>
      <div style={{ color: '#6B7280', fontSize: 14, marginBottom: 20 }}>{sub
        {children}
      </div>
    </div>
  );
}
```

Three props (title, subtitle, children). The chart goes inside via the children prop:

```

<ChartCard title="Points Breakdown by Round" subtitle="How each team has scored" >
  <ResponsiveContainer width="100%" height={360}>
    <BarChart data={r1Data}>
      <CartesianGrid />
      <XAxis dataKey="team" />
      <YAxis />
      <Tooltip />
      <Legend />
      <Bar dataKey="R1" fill="#0F5132" />
    </BarChart>
  </ResponsiveContainer>
</ChartCard>

```

This is the **layout-vs-content separation** from Chapter 5 in action. The card knows about title, subtitle, padding, shadow. The chart knows about bars and axes. **Separated concerns; reusable card.**

Custom tooltip styling

```

<Tooltip
  contentStyle={{
    background: '#FFFBEB',
    border: '2px solid #FBBF24',
    borderRadius: 8,
    fontSize: 14,
  }}
/>

```

You can pass a `contentStyle` object to style the default tooltip box. Works for most needs.

For more elaborate tooltips (custom layout, conditional content), pass a `content render` prop:

```

<Tooltip content={({ active, payload, label }) => {
  if (!active || !payload || !payload.length) return null;
  return (

```

```

    <div style={{ background: '#FFF', padding: 8, border: '1px solid #ccc' }}
      <strong>{label}</strong>
      {payload.map(p => (
        <div key={p.name} style={{ color: p.color }}>{p.name}: {p.value}</div>
      ))}
    </div>
  );
}} />

```

We don't bother in our codebase — defaults are fine.

Bar chart vs line chart vs area chart

Same data shape, different component:

```

<LineChart data={r1Data}>
  <Line dataKey="R1" stroke="#0F5132" />
  <Line dataKey="R2" stroke="#DC2626" />
</LineChart>

<AreaChart data={r1Data}>
  <Area dataKey="R1" stroke="#0F5132" fill="#0F5132" fillOpacity={0.3} />
  <Area dataKey="R2" stroke="#DC2626" fill="#DC2626" fillOpacity={0.3} />
</AreaChart>

```

The data array is identical. The chart wrapper changes. **Chart type is a thin layer over the same data.** Once you've reshaped your data correctly, switching chart types is a one-line edit.

When NOT to use Recharts

Recharts is great for the bread-and-butter dashboards — bars, lines, pies, radar charts. It struggles with:

- **Highly custom interactions** (drag a region, lasso-select points, etc.). Use D3 directly or Visx.
- **Tens of thousands of data points** at once. Recharts isn't a perf monster; for canvas-rendered high-data charts, use [uPlot](#) or pull from D3 directly.

- **Animations beyond Recharts' built-ins.** The animation API is limited.
- **Geographic / map visualizations.** Use [react-leaflet](#) or [react-map-gl](#).

For our snooker app, Recharts is the right tool. We're drawing 8 bars, not 80,000.

A worked example: simplest possible chart

Suppose you want to render a single bar chart of total points per team. The reshape:

```
const data = teams.map(t => ({ name: t.name, total: t.scores.total }));
```

The chart:

```
<ChartCard title="Total Points" subtitle="">
  <ResponsiveContainer width="100%" height={360}>
    <BarChart data={data}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="name" />
      <YAxis />
      <Tooltip />
      <Bar dataKey="total" fill="#0F5132" />
    </BarChart>
  </ResponsiveContainer>
</ChartCard>
```

Eight rows, each with { name, total }. One <Bar> reading total. Done. **Eight lines of JSX, ~3 lines of data shaping.**

That's the baseline. Lessons 2 and 3 build on this with stacks, cell colors, and pivots.

A note on <RechartsCommonProps>

A small TypeScript wrinkle: most Recharts components accept dozens of props. The TS types reflect this — autocomplete shows you 40+ options. **Most are optional.** Stick to the ones you've seen in this lesson; the rest are advanced.

Vibe prompt you would have used (for the simplest chart)

"Create an Analytics tab at components/tabs/AnalyticsTab.tsx. First chart: a bar chart of total points per team using Recharts. Reshape teams.map(t => ({ name: t.name, total: t.scores.total })). Wrap in a <ChartCard> (which I'll write next) with title 'Total Points'. Use ResponsiveContainer (100% width, 360 height), CartesianGrid (3 3 dasharray, light yellow), XAxis dataKey='name', YAxis defaults, Tooltip with contentStyle (cream bg, gold border), one <Bar dataKey='total' fill='#0F5132'>. Tree-shake imports from 'recharts'. Don't add any other charts yet."

Specific, scoped, names every visual decision. **You'll get exactly this chart and nothing else.**

CHECK YOURSELF

1. **Recharts charts render at 0 pixels without <ResponsiveContainer>. Why?** What is the container actually doing?
2. **Two <Bar> elements with the same stackId produce a stacked bar. With different stackIds, what do they produce?**
3. **<XAxis dataKey="team"> — what happens if your data array has a typo and the field is actually called teamName?** Walk through what shows on the X-axis.
4. **Why is <Tooltip> a separate component instead of a prop on <BarChart>?** What architectural pattern does this reflect?
5. **You decide to switch from a stacked bar chart to a line chart with the same data. What's the minimum change?**

When you've answered these, head to [02-reshaping-data.md](#).